```
import React, { useState, useRef, useMemo, useCallback } from "react";

// Child component that only re-renders when props change
const Child = React.memo(({ handleClick, result }) => {
  console.log("Child Rendered"); // To see when child re-renders
  return (
    <div>
      <button onClick={handleClick}>Increment</button>
      <p>Memoized Result: {result}</p>
    </div>
  );
});

function App() {
  const [count, setCount] = useState(0);
  const inputRef = useRef(null); // useRef to access DOM input
  const [multiplier, setMultiplier] = useState(1);

  // useCallback to memoize the increment function
  const handleIncrement = useCallback(() => {
    setCount(prev => prev + 1);
    if (inputRef.current) {
      console.log("Input Value:", inputRef.current.value);
    }
  }, []);

  // useMemo to compute expensive calculation only when dependencies change
```

```
const computedValue = useMemo(() => {
  console.log("Calculating memoized result...");
  return count * multiplier;
}, [count, multiplier]);

return (
  <div style={{ padding: "20px" }}>
    <h2>useRef, useMemo, useCallback Demo</h2>

    <input
      ref={inputRef}
      type="text"
      placeholder="Type something here..."
      style={{ marginBottom: "10px" }}
    />

    <div>
      <label>
        Multiplier:
        <input
          type="number"
          value={multiplier}
          onChange={(e) => setMultiplier(Number(e.target.value))}
          style={{ marginLeft: "5px" }}
        />
      </label>
    </div>
```

```
      <p>Count: {count}</p>


      <Child handleClick={handleIncrement} result={computedValue} />
    </div>
  );
}


export default App;
```